S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 07

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Name: Sudhanshu K. Gedam

USN:CM24051

Semester / Year:

Academic Session:

Date of Performance:

Date of Submission:

- Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

- Objectives:
  1. Implement the Banker's Algorithm to manage resource allocation and prevent deadlocks.
  2. Calculate the current resource need for each process based on allocated and maximum resources.
  3. Determine a safe execution sequence for processes if resources are available.

- Requirements:
  - Hardware Requirements:
    - Processor: Minimum 1 GHz
    - RAM: 512 MB or higher
    - Storage: 100 MB free space
  - Software Requirements:
    - Operating System: Linux/Unix-based
    - Shell: Bash 4.0 or higher
    - Text Editor: Nano, Vim, or any preferred editor

- Theory:

Banker’s Algorithm in Operating System

Banker’s Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It ensures that a system remains in a safe state by carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks.

- The Banker’s Algorithm is a smart way for computer systems to manage how programs use resources, like memory or CPU time.
- It helps prevent situations where programs get stuck and can not finish their tasks. This condition is known as deadlock.
- By keeping track of what resources each program needs and what’s available, the banker algorithm makes sure that programs only get what they need in a safe order.

Why Banker's Algorithm is Named So?

The banker's algorithm is named so because it is used in the banking system to check whether a loan can be sanctioned to a person or not. Suppose there are n number of account holders in a bank and the total sum of their money is S. Let us assume that the bank has a certain amount of money Y. If a person applies for a loan then the bank first subtracts the loan amount from the total money that the bank has (Y) and if the remaining amount is greater than S then only the loan is sanctioned. It is done because if all the account holders come to withdraw their money then the bank can easily do it.

It also helps the OS to successfully share the resources between all the processes. The banker's algorithm uses the notation of a safe allocation state to ensure that granting a resource request cannot lead to a deadlock either immediately or in the future. In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers.

The bank would try to be in a safe state always.

Components of the Banker's Algorithm

The following Data structures are used to implement the Banker's Algorithm:

Let 'n' be the number of processes in the system and 'm' be the number of resource types.

1. Available
   - It is a 1-D array of size 'm' indicating the number of available resources of each type.
   - Available[j] = k means there are 'k' instances of resource type Rj
2. Max
   - It is a 2-d array of size 'n*m' that defines the maximum demand of each process in a system.
   - Max[i, j] = k means process Pi may request at most 'k' instances of resource type Rj.
3. Allocation
   - It is a 2-d array of size 'n*m' that defines the number of resources of each type currently allocated to each process.
   - Allocation[i, j] = k means process Pi is currently allocated 'k' instances of resource type Rj
4. Need
   - It is a 2-d array of size 'n*m' that indicates the remaining resource need of each process.
   - Need [i, j] = k means process Pi currently needs 'k' instances of resource type Rj
   - Need [i, j] = Max [i, j] – Allocation [i, j]

Allocation specifies the resources currently allocated to process Pi and Need specifies the additional resources that process Pi may still request to complete its task.

Banker's algorithm consists of a Safety algorithm and a Resource request algorithm.

Banker's Algorithm

1. Active := Running U Blocked for k = 1 to r:
New_request[k] := Requested_resources[requesting_process, k]
2. Simulated_allocation := Allocated_resources for k
= 1 to r: // Compute projected allocation state
Simulated_allocation[requesting_process, k] :=
Simulated_allocation[requesting_process, k] + New_request[k]
3. Feasible := true for k = 1 to r: // Check if projected
allocation state is feasible if Total_resources[k] <
Simulated_total_alloc[k]:
Feasible := false 4. if
Feasible = true:
while set Active contains a process P1 such that:
for all k, Total_resources[k] – Simulated_total_alloc[k] >=
Max_need[P1, k] – Simulated_allocation[P1, k]: Delete P1
from Active for k = 1 to r:
Simulated_total_alloc[k] :=
Simulated_total_alloc[k] – Simulated_allocation[P1, k]
5. if set Active is empty:
for k = 1 to r: // Delete the request from pending requests
Requested_resources[requesting_process, k] := 0 for k =
1 to r: // Grant the request
Allocated_resources[requesting_process, k] :=
Allocated_resources[requesting_process, k] + New_request[k]
Total_alloc[k] := Total_alloc[k] + New_request[k]

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1. Let Work and Finish be vectors of length 'm' and 'n' respectively.
   Initialize: Work = Available
   Finish[i] = false; for i=1, 2, 3, 4….n
2. Find an i such that both
   Finish[i] = false 2.2
   Needi <= Work
   if no such i exists goto step (4)
3. Work = Work + Allocation[i] Finish[i] = true goto step (2)
4. if Finish [i] = true for all i then the system is in a safe state.

Resource-Request Algorithm

Let Requesti be the request array for process Pi. Requesti Li] = k means process Pi wants k instances of resource type Rj. When a request for resources is made by process Pi, the following actions are taken:

1. If Requesti <= Needi

Goto step (2) ; otherwise, raise an error condition, smce the process has exceeded its maximum claim.

2. If Requesti <= Available

Goto step (3); otherwise, Pi must wait, since the resources are not available.

3. Have the system pretend to have allocated the requested resources to process Pi by modifying the state as follows:

Available= Available - Requesti

Allocationi = Allocationi + Requesti

Needi = Needi- Requesti

Example: Considering a system with five processes P0 through P4 and three resources of type A, **B,** C. Resource type A has 10 instances, **B** has 5 instances and type C has 7 instances. Suppose at time t0 following snapshot of the system has been taken:

Available:

A	B	C
7	2	3

Process	Allocation			Max			Need		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	4	2	2	2	1	1
P4	0	0	2	5	3	3	5	3	1

Q.1 What will be the content of the Need matrix?

Need [i, j] = Max [i, j] - Allocation [i, j]

So, the content of Need Matrix is:

What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?

Unsafe State in The Bankers Algorithm

In the context of the Banker's Algorithm, an unsafe state refers to a situation in which, all processes in a system currently hold resources within their maximum needs, there is no guarantee that the system can avoid a deadlock in the future. This state is contrasted with a safe state, where there exists a sequence of resource allocation and deallocation that guarantees that all processes can complete without leading to a deadlock.

- Key Concepts in Banker's Algorithm

  Safe State: There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.
- Unsafe State: Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.

  Example of Unsafe State

  Consider a system with three processes (P1, P2, P3) and 6 instances of a resource. Let's say:
- The available instances of the resource are: 1
- The current allocation of resources to processes is:
  - P1: 2
  - P2: 3
  - P3: 1
- The maximum demand (maximum resources each process may eventually request) is:
  - P1: 4
  - P2: 5
  - P3: 3

Department of Emerging Technologies, SBJITMR, Nagpur.

In this situation:
Need = Maximum Demand – Allocation

Process	Maximum Demand	Allocation	Need
P1	4	2	2
P2	5	3	2
P3	3	1	2

- P1 may need 2 more resources to complete.
- P2 may need 2 more resource to complete.
- P3 may need 2 more resources to complete.

However, there is only 1 resource available. Even though none of the processes are currently deadlocked, the system is in an unsafe state because there is no sequence of resource allocation that guarantees all processes can complete.

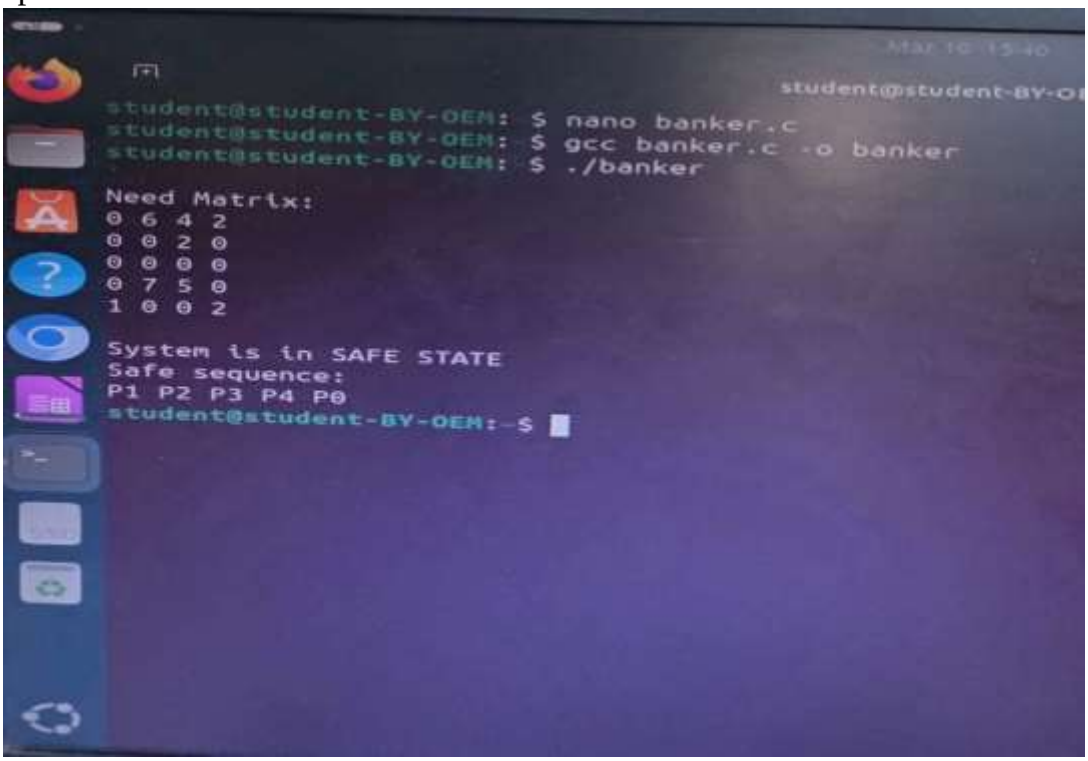
□ CODE:

```
#include <stdio.h>

int main() {        int
processes = 5;
   int resources = 4;

   int allocation[5][4] = {
      {0,0,1,4},
      {0,6,3,2},
      {0,0,1,2},
      {1,0,0,0},
      {1,3,5,4}
   };

   int max[5][4] = {
      {0,6,5,6},
      {0,6,5,2},
      {0,0,1,2},
      {1,7,5,0},
      {2,3,5,6}
   };

   int available[4] = {1,6,2,0};
```

```
    int work[4];

    for(int i=0;i<resources;i++)
        work[i] = available[i];

    printf("\nNeed Matrix:\n");

    for(int i=0;i<processes;i++){          for(int
j=0;j<resources;j++){              need[i][j] =
max[i][j] - allocation[i][j];              printf("%d
",need[i][j]);
        }
        printf("\n");
    }

    int count=0;

    while(count<processes){
        int found = 0;

        for(int i=0;i<processes;i++){

            if(finish[i]==0){

                int j;

                for(j=0;j<resources;j++){
if(need[i][j] > work[j])
                        break;
                }

                if(j==resources){

                    for(int k=0;k<resources;k++)
work[k] += allocation[i][k];

                    safeSeq[count++] = i;
                    finish[i] = 1;
found = 1;
                }
```

```
            }
        }

        if(found==0){

            printf("\nSystem is NOT in safe state\n");
  return 0;
        }
    }

    printf("\nSystem is in SAFE STATE\nSafe sequence:\n");

    for(int i=0;i<processes;i++)
        printf("P%d ",safeSeq[i]);

    printf("\n");

    return 0;
     }
```

□ Output:



- Conclusion: In this practical, we conclude that the Banker's Algorithm effectively ensures safe resource allocation by preventing deadlocks. By calculating the current resource need of each process and checking for a safe execution sequence, we can determine whether all processes can complete execution without system instability.

- Discussion Questions:

1. What is the purpose of the Banker's Algorithm?
2. How do you calculate the current need of each process?
3. What are the conditions for a system to be in a safe state?
4. What happens if no safe sequence exists in the Banker's Algorithm?
5. What are the key inputs required to implement the Banker's Algorithm?

- References:

https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system-2/

https://chatgpt.com/

Date: / /2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26

S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 08

Aim: A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Least recently used page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

Name: Sudhanshu K. Gedam

USN: CM24051

Semester / Year:

Academic Session:

Date of Performance:

Date of Submission:

Aim: A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Least recently used page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

□ Objectives:

1. Implement the Least Recently Used (LRU) page replacement algorithm for memory management.
2. Calculate the number of page hits and page faults for a given page reference sequence.
3. Determine the percentage of page hits and page faults based on the system's frame capacity.

□ Requirements:

□ Hardware Requirements:

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

□ Software Requirements:

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

□ Theory:

In an operating system, page replacement refers to a scenario in which a page from the main memory should be replaced by a page from the secondary memory. Page replacement occurs due to page faults. The various page replacement algorithms like FIFO, Optimal page replacement, LRU, LIFO, and Random page replacement help the operating system decide which page to replace.

What is Page Replacement in Operating Systems?

Page replacement is needed in the operating systems that use virtual memory using Demand Paging. As we know in Demand paging, only a set of pages of a process is loaded into the memory. This is done so that we can have more processes in the memory at the same time.

When a page that is residing in virtual memory is requested by a process for its execution, the Operating System needs to decide which page will be replaced by this requested page. This process is known as page replacement and is a vital component in virtual memory management.

Why Need Page Replacement Algorithms?

To understand why we need page replacement algorithms, we first need to know about page faults. Let's see what is a page fault.

Page Fault: A Page Fault occurs when a program running in CPU tries to access a page that is in the address space of that program, but the requested page is currently not loaded into the main physical memory, the RAM of the system. Since the actual RAM is much less than the virtual memory the page faults occur. So whenever a page fault occurs, the Operating system has to replace an existing page in RAM with the newly requested page. In this scenario, page replacement algorithms help the Operating System in deciding which page to replace. The primary objective of all the page replacement algorithms is to minimize the number of page faults.

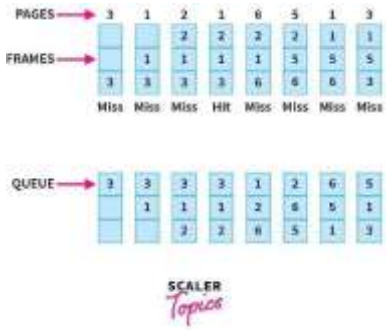
Page Replacement Algorithms in Operating Systems

First In First Out (FIFO)

The FIFO algorithm is the simplest of all the page replacement algorithms. In this, we maintain a queue of all the pages that are in the memory currently. The oldest page in the memory is at the front end of the queue and the most recent page is at the back or rear end of the queue.

Whenever a page fault occurs, the operating system looks at the front end of the queue to know the page to be replaced by the newly requested page. It also adds this newly requested page at the rear end and removes the oldest page from the front end of the queue.

Example: Consider the page reference string as 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames. Let's try to find the number of page faults:



Initially, all of the slots are empty so page faults occur at 3,1,2. Page faults = 3
When page 1 comes, it is in the memory so no page fault occurs.
Page faults = 3
When page 6 comes, it is not present and a page fault occurs. Since there are no empty slots, we remove the front of the queue, i.e 3.
Page faults = 4
When page 5 comes, it is also not present, and hence a page fault occurs. The front of the queue i.e. 1 is removed.
Page faults = 5
When page 1 comes, it is not found in memory and again a page fault occurs. The front of the queue i.e. 2 is removed.
Page faults = 6
When page 3 comes, it is again not found in memory, a page fault occurs, and page 6 is removed being on top of the queue Total page faults = 7

Belady's anomaly: Generally if we increase the number of frames in the memory, the number of page faults should decrease due to obvious reasons. Belady's anomaly refers to the phenomena where increasing the number of frames in memory, increases the page faults as well.

Advantages

- Simple to understand and implement.
- Does not cause more overhead.

Disadvantages

- Poor performance
- Doesn't use the frequency of the last used time and just simply replaces the oldest page.
- Suffers from Belady's anomaly.
- Optimal Page Replacement in OS

Optimal page replacement is the best page replacement algorithm as this algorithm results in the least number of page faults. In this algorithm, the pages are replaced with the ones that will not be used for the longest duration of time in the future. In simple terms, the pages that will be referred to farthest in the future are replaced in this algorithm.

Example:

Let's take the same page reference string 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames as we saw in FIFO. This also helps you understand how Optimal Page replacement works the best.

Initially, since all the slots are empty, pages 3, 1, 2 cause a page fault and take the empty slots.
Page faults = 3
When page 1 comes, it is in the memory and no page fault occurs.
Page faults = 3
When page 6 comes, it is not in the memory, so a page fault occurs and 2 is removed as it is not going to be used again.
Page faults = 4
When page 5 comes, it is also not in the memory and causes a page fault. Similar to above 6 is removed as it is not going to be used again.
page faults = 5
When page 1 and page 3 come, they are in the memory so no page fault occurs.
Total page faults = 5

Advantages

- Excellent efficiency.
- Less complexity.
- Easy to use and understand.
- Simple data structures can be used to implement.
- Used as the benchmark for other algorithms.

Disadvantages

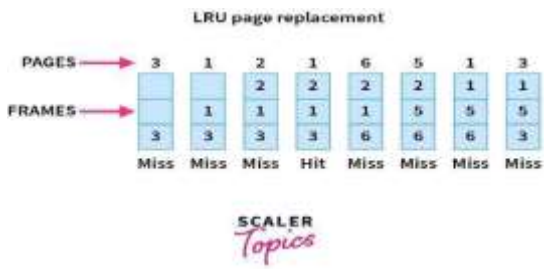
- More time consuming
- Difficult for error handling
- Need future awareness of the programs, which is not possible every time

Least Recently Used (LRU) Page Replacement Algorithm

The least recently used page replacement algorithm keeps the track of usage of pages over a period of time. This algorithm works on the basis of the principle of locality of a reference which states that a program has a tendency to access the same set of memory locations repetitively over a short period of time. So pages that have been used heavily in the past are most likely to be used heavily in the future also.

In this algorithm, when a page fault occurs, then the page that has not been used for the longest duration of time is replaced by the newly requested page.

Example: Let's see the performance of the LRU on the same reference string of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames:



Initially, since all the slots are empty, pages 3, 1, 2 cause a page fault and take the empty slots.
Page faults = 3
When page 1 comes, it is in the memory and no page fault occurs.
Page faults = 3
When page 6 comes, it is not in the memory, so a page fault occurs and the least recently used page 3 is removed.
Page faults = 4
When page 5 comes, it again causes a page fault, and page 1 is removed as it is now the least recently used page.
Page faults = 5
When page 1 comes again, it is not in the memory, and hence page 2 is removed according to the LRU.
Page faults = 6

When page 3 comes, the page fault occurs again and this time page 6 is removed as the least recently used one.
Total page faults = 7

Now in the above example, the LRU causes the same page faults as the FIFO, but this may not always be the case as it will depend upon the series, the number of frames available in memory, etc. In fact, on most occasions, LRU is better than FIFO.

Advantages

- It is open for full analysis
- Doesn't suffer from Belady's anomaly
- Often more efficient than other algorithms

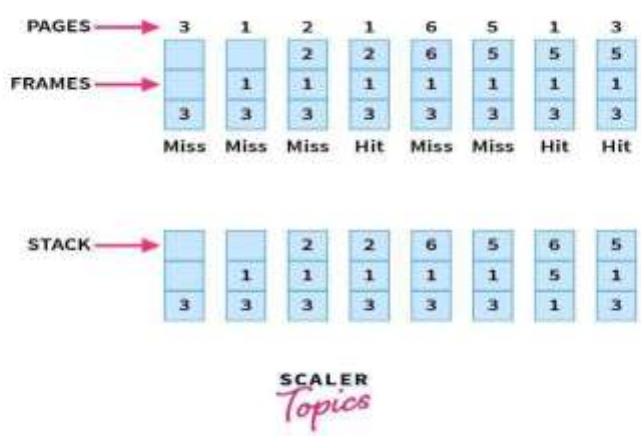
Disadvantages

- It requires additional data structures to be implemented
- More complex
- High hardware assistance is required.

Last In First Out (LIFO) Page Replacement Algorithm

This is the Last in First Out algorithm and works on LIFO principles. In this algorithm, the newest page is replaced by the requested page. Usually, this is done through a stack, where we maintain a stack of pages currently in the memory with the newest page being at the top. Whenever a page fault occurs, the page at the top of the stack is replaced.

Example: Let's see how the LIFO performs for our example string of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames:



Initially, since all the slots are empty, page 3,1,2 causes a page fault and takes the empty slots.

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs.

Page faults = 3

When page 6 comes, the page fault occurs and page 2 is removed as it is on the top of the stack and is the newest page.

Page faults = 4

When page 5 comes, it is not in the memory, which causes a page fault, and hence page 6 is removed being on top of the stack.

Page faults = 5

When page 1 and page 3 come, they are in memory already, hence no page fault occurs.

Total page faults = 5

As you may notice, this is the same number of page faults as the Optimal page replacement algorithm. So we can say that for this series of pages, this is the best algorithm that can be implemented without the prior knowledge of future references.

Advantages

- Simple to understand
- Easy to implement
- No overhead Disadvantages

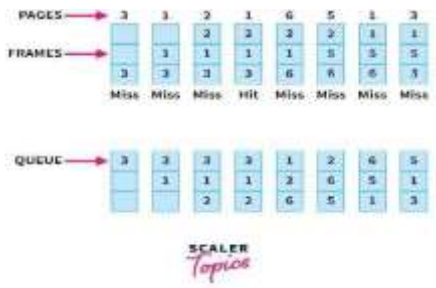
Does not consider Locality principle, hence may produce worst performance.

The old pages may reside in memory forever even if they are not used.

Random Page Replacement in OS

This algorithm, as the name suggests, chooses any random page in the memory to be replaced by the requested page. This algorithm can behave like any of the algorithms based on the random page chosen to be replaced.

Example: Suppose we choose to replace the middle frame every time a page fault occurs. Let’s see how our series of 3, 1, 2, 1, 6, 5, 1, 3 with 3-page frames perform with this algorithm:



Initially, since all the slots are empty, page 3,1,2 causes a page fault and takes the empty slots

Page faults = 3

When page 1 comes, it is in the memory and no page fault occurs.

Page faults = 3

When page 6 comes, the page fault occurs, we replace the middle element i.e 1 is removed.

Page faults = 4

When page 5 comes, the page fault occurs again and middle element 6 is removed

Page faults = 5

When page 1 comes, there is again a page fault, and again the middle element 5 is removed

Page faults = 6

When page 3 comes, it is in memory, hence no page fault occurs.

Total page faults = 6

As we can see, the performance is not the best, but it's also not the worst. The performance in the random replacement algorithm depends on the choice of the page chosen at random.

Advantages

Advantages

- Easy to understand and implement

No extra data structure needed to implement

- No overhead

Disadvantages

- Can not be analyzed, may produce different performances for the same series.
- Can suffer from Belady's anomaly.

Summary:

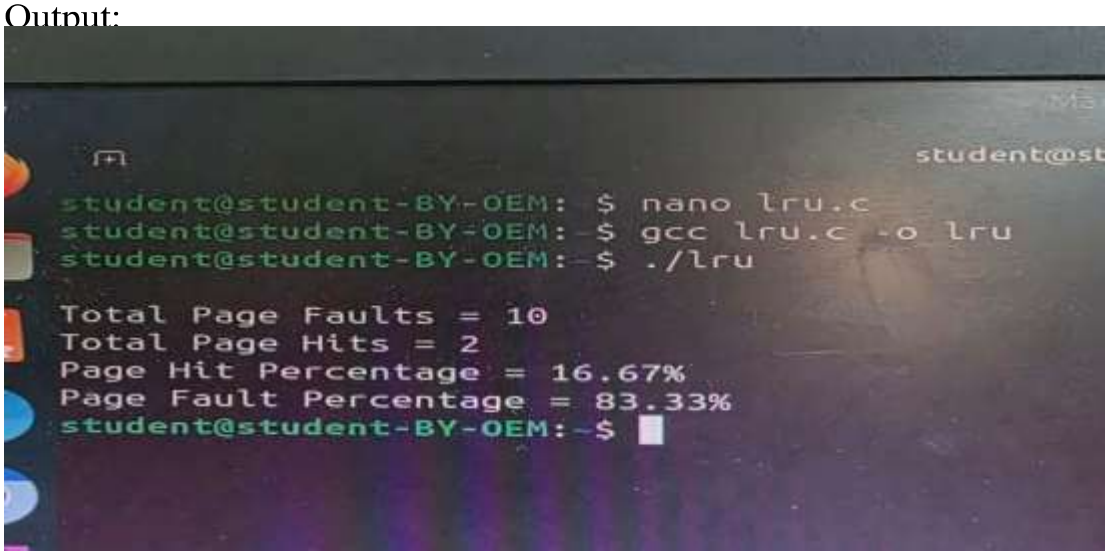
i. The objective of page replacement algorithms is to minimize the page faults
ii. FIFO page replacement algorithm replaces the oldest page in the memory
iii. Optimal page replacement algorithm replaces the page which will be referred farthest in the future
iv. LRU page replacement algorithm replaces the page that has not been used for the longest duration of time
v. LIFO page replacement algorithm replaces the newest page in memory
vi. Random page replacement algorithm replaces any page at random
vii. Optimal page replacement algorithm is considered to be the most effective algorithm but cannot be implemented in practical scenarios due to various limitations

□ CODE:

```
#include <stdio.h>

int main() {

   int pages[] = {1,2,3,4,1,2,5,1,2,3,4,5};
int n = 12;
   int frames = 3;

   int  frame[10],  time[10];            int
i,j,pos,faults=0,hits=0,counter=0;

   for(i=0;i<frames;i++)
frame[i] = -1;
```

```
            {
                counter++;
time[j]=counter;
                hits++;
flag=1;                break;
            }
        }

        if(flag==0)
        {
            int min=999;
            for(j=0;j<frames;j++)
            {
                if(frame[j]==-1)
                {
pos=j;
break;
                }

                if(time[j]<min)
                {
                    min=time[j];
pos=j;
                }
            }

            frame[pos]=pages[i];
counter++;            time[pos]=counter;
            faults++;
        }
    }
    printf("\nTotal Page Faults = %d",faults);
    printf("\nTotal Page Hits = %d",hits);

    float hit_per = (hits*100.0)/n;
    float fault_per = (faults*100.0)/n;

    printf("\nPage Hit Percentage = %.2f%%",hit_per);
printf("\nPage Fault Percentage = %.2f%%\n",fault_per);
```

- Output:



- Conclusion: In this practical, we conclude that the LRU page replacement algorithm helps manage memory efficiently by minimizing page faults and improving performance .

D Discussion Questions:

1. What is the Least Recently Used (LRU) page replacement algorithm?
2. How do you determine a page hit and a page fault?
3. What is the formula to calculate page hit and page fault percentage?
4. Why is LRU preferred over FIFO for page replacement?
5. What is the worst-case scenario for LRU page replacement?

- References:

https://www.scaler.com/topics/operating-system/page-replacement-algorithm/
https://chatgpt.com/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26